Centro Universitário IESB
Curso: Ciência da Computação
Disciplina: Programação Paralela (CCO085)
Semestre: 2026/2 · Campus Asa Sul
Turmas: CCODM3B (matutino) e CCONM3B (noturno)

Orientações do Projeto Acadêmico

Paralelização de um Problema Computacionalmente Intensivo

Prof. Rodrigo Gonçalves

1. Finalidade do Projeto

O Projeto Acadêmico da disciplina de Programação Paralela tem como finalidade consolidar, de forma prática e técnica, os conteúdos estudados ao longo do semestre. O objetivo principal não é apenas escrever código paralelo, mas demonstrar compreensão conceitual e domínio das decisões de implementação sobre um programa que roda de verdade e acelera de verdade.

O problema e a **implementação sequencial de referência são fornecidos pelo professor**. Você não vai escolher nem escrever o algoritmo sequencial — ele já está pronto, correto e instrumentado. O trabalho desta disciplina é o passo seguinte: **transformar esse código sequencial em versões paralelas que produzam o mesmo resultado e executem em menos tempo**, medindo rigorosamente o ganho obtido em cada etapa e explicando por que o ganho real difere do ganho teórico.

O projeto deverá integrar obrigatoriamente:

- Baseline e perfilamento: medição do tempo de referência e identificação dos pontos quentes (hotspots);
- Paralelismo em memória compartilhada (OpenMP), com tratamento de condições de corrida;
- Paralelismo em memória distribuída (MPI), com decomposição e comunicação entre processos;
- Análise experimental: speedup, eficiência, escalabilidade e confronto com a Lei de Amdahl.

Cada trio recebe **um problema diferente** de uma lista de dez (Seção 4), com o respectivo código sequencial. O resultado que esse código produz para uma dada entrada é o *gabarito*: as suas versões paralelas têm que chegar exatamente ao mesmo resultado.

2. Organização dos Grupos

- A turma será dividida em **grupos de 3 alunos**, de livre formação. Grupos incompletos serão ajustados pelo professor;
- Cada integrante assumirá uma **responsabilidade técnica formal** (descrita na Seção 3);
- **Todos os integrantes devem compreender integralmente o projeto.** Durante a apresentação, cada integrante será questionado individualmente — e, como detalha a Seção 6, a questão sorteada será sobre a frente de *outro* colega;
- Cada trio escolhe seu problema da lista da Seção 4. **Não é permitido que dois trios da mesma turma peguem o mesmo problema**; a atribuição é por ordem de escolha, até a aula de 27/08.

3. Responsabilidades Individuais Obrigatórias

Cada grupo deverá formalmente designar as seguintes funções (uma por aluno). A divisão espelha o caminho natural de uma paralelização: primeiro entender onde está o custo, depois atacar em memória compartilhada, e por fim em memória distribuída. Cada frente produz um artefato que **roda e pode ser verificado na hora**.

1. Responsável pelo Baseline e Perfilamento

- Estudo do código sequencial de referência: análise de complexidade e identificação dos pontos quentes;
- Perfilamento com evidência (tabela de tempo por trecho ou função) que aponte onde o programa gasta o tempo;
- Estimativa da fração paralelizável e do speedup máximo previsto pela **Lei de Amdahl**;
- Definição do protocolo de medição (número de repetições, descarte de aquecimento, tamanhos de entrada).

2. Responsável pela Versão OpenMP (memória compartilhada)

- Implementação em OpenMP a partir do baseline, escolhendo o padrão adequado (laço paralelo, seções, tasks);
- Tratamento explícito de **condições de corrida** e escolha justificada entre `critical`, `atomic` e `reduction`;
- Estudo comparativo das políticas de escalonamento (`static`, `dynamic`, `guided`) quando o problema tiver carga desigual;
- Curva de **speedup por número de threads** (1, 2, 4, 8...), com o resultado validado contra o baseline.

3. Responsável pela Versão MPI (memória distribuída)

- Implementação em MPI a partir do baseline, com **decomposição** de domínio ou de tarefas justificada;
- Uso de pelo menos uma **operação coletiva** (broadcast, scatter, gather, reduce ou allreduce);
- Análise do **overhead de comunicação** em função do número de processos;
- Curva de speedup por número de processos (`-np` 1, 2, 4...), com o resultado validado contra o baseline.

> Cada frente entrega **um código que executa e uma medida que existe**. É de propósito: o professor corrige *rodando*, não apenas lendo. Um baseline que não roda, um OpenMP que não acelera ou um MPI cujo resultado muda com o número de processos são falhas objetivas, visíveis na hora.

Observação: embora cada aluno tenha uma responsabilidade principal, todos devem ter visão do projeto completo. Durante a apresentação, o professor questionará cada integrante individualmente sobre uma frente que não é a sua.

4. Lista de Problemas

Cada trio escolhe **um** problema desta lista; o código sequencial de referência de cada um é fornecido pelo professor. Dois trios da mesma turma não podem escolher o mesmo problema.

Nº	Problema	Aspecto que o torna interessante
1	Multiplicação de matrizes densas	Efeito de cache e da ordem dos laços; comparação com o pico teórico
2	Estimativa de π por Monte Carlo	Paralelismo quase perfeito; geração de aleatórios independentes por thread

Nº	Problema	Aspecto que o torna interessante
3	Conjunto de Mandelbrot	Desbalanceamento natural de carga; motiva `schedule(dynamic)`
4	Filtro de convolução 2D (blur)	Decomposição de domínio 2D; bordas e troca de halo
5	Equação do calor 2D	Stencil iterativo com troca de bordas a cada passo; caso clássico de MPI
6	Simulação de N corpos	Custo quadrático; comunicação global a cada passo
7	Ordenação (mergesort)	Divisão e conquista; alvo natural de `tasks` em OpenMP
8	Contagem de k-mers	Redução sobre histograma; particionamento de dados
9	Busca exaustiva (força bruta)	Mestre-escravo; efeito do término antecipado na medição
10	Agrupamento por k-médias	Fases paralelas alternadas com sincronização global

5. Entregáveis e Requisitos Técnicos

5.1 Marco 1 — entrega em 10/09/2026 (vale 2,0 pontos na média A1)

Documento em PDF (até 6 páginas) e código, contendo:

- Identificação do trio e do problema escolhido;
- Análise do algoritmo sequencial fornecido: complexidade assintótica e explicação do que ele faz;
- **Perfilamento**: tabela do tempo gasto nos principais trechos, identificando os hotspots;
- **Baseline medido**: tempo de execução para pelo menos três tamanhos de entrada, com a metodologia descrita (número de repetições, descarte de aquecimento, máquina utilizada);
- **Fração paralelizável estimada** e speedup máximo previsto pela Lei de Amdahl;
- **Plano de paralelização**: qual padrão será usado em OpenMP e em MPI, e por quê.

5.2 Marco 2 — entrega em 12/11/2026 (compõe os 4,0 pontos de artefato da A2)

Entregue antes da primeira data de apresentação, para que nenhum grupo ganhe tempo extra por ser escalado depois. Deve conter:

- **Versão OpenMP** funcionando, com tratamento de condições de corrida e curva de speedup por threads;
- **Versão MPI** funcionando, com decomposição, operação coletiva e curva de speedup por processos;
- **Validação de corretude**: prova de que as duas versões produzem o mesmo resultado do baseline;
- **Análise experimental**: gráficos de tempo, speedup e eficiência; escalabilidade forte e fraca; e a discussão do afastamento em relação à Lei de Amdahl (overhead de comunicação, desbalanceamento de carga, false sharing, limite de banda de memória).

5.3 Formato de entrega

Repositório Git (público ou com acesso ao professor), contendo o código das três versões (sequencial, OpenMP, MPI) em diretórios separados, um `Makefile` que compile tudo no ambiente da disciplina, um script que gere os dados de medição em CSV, um `README.md` que rode o projeto do zero, os gráficos e um `AUTORES.md` declarando a frente de cada integrante. O histórico de commits faz parte da avaliação: entrega em commit único, ou concentrada numa só pessoa na véspera, é penalizada.

6. Composição da Nota

Primeiro bimestre (A1): Prova P1 escrita e individual, 8,0 + Marco 1 do projeto, 2,0.

Segundo bimestre (A2) — formato híbrido: Artefato do projeto (Marco 2), 4,0, nota do grupo + Apresentação e arguição, 6,0.

Média final: MF = 0,4 × A1 + 0,6 × A2. Aprovação com MF $\geq$ 5,0 e frequência $\geq$ 75%. Quem ficar abaixo de 5,0 realiza a P3 (10/12), prova escrita individual sobre todo o conteúdo, que substitui a média menos favorável.

7. Seminário Técnico e Arguição Cruzada

O projeto será apresentado em formato de seminário técnico, com duração aproximada de 18 minutos por grupo, em uma de duas datas:

- Bloco 1: 19/11/2026
- Bloco 2: 26/11/2026

A escala com o grupo de cada data é divulgada em 05/11. A apresentação divide-se em:

- **Demonstração ao vivo (~9 min):** o programa rodando na máquina do laboratório, com cada integrante mostrando a sua frente — o baseline, a aceleração em OpenMP e a execução em MPI. **Não é permitido vídeo gravado ou captura de tela** — o programa tem que executar na hora, e o speedup tem que aparecer numa máquina com vários núcleos;
- **Arguição individual (~9 min):** cada integrante é sorteado para responder uma questão do banco público de perguntas.

Regra central — a arguição é cruzada. Cada integrante será sorteado para responder uma questão sobre a frente de *outro* colega, não sobre a sua. O responsável pelo MPI pode ser questionado sobre por que a curva de OpenMP satura; o responsável pelo baseline pode ter que explicar a escolha de uma operação coletiva no MPI. Ninguém sabe de antemão qual frente vai defender. O motivo é simples: a divisão de trabalho serve para vocês aprenderem a colaborar, não para cada um se especializar num canto e ignorar o resto do sistema. Ao final da disciplina, cada um precisa entender o projeto inteiro — e é isso que a arguição cruzada verifica. Não adianta cada um cuidar só da sua parte: o grupo precisa se reunir e revisar o trabalho um do outro.

A demonstração deverá contemplar, no mínimo: o baseline rodando e produzindo o resultado de referência; a versão OpenMP acelerando e mantendo o mesmo resultado; a versão MPI rodando com vários processos e mantendo o mesmo resultado; e pelo menos um gráfico de speedup construído a partir dos dados medidos.

8. Avaliação do Projeto

8.1 Artefato do grupo — 4,0 pontos (nota compartilhada)

Critério	Pontos
Baseline, perfilamento e estimativa de Amdahl	1,0
Versão OpenMP correta, com tratamento de corrida e speedup medido	1,0
Versão MPI correta, com decomposição e coletiva	1,0
Análise experimental (gráficos, escalabilidade, discussão) e documentação	1,0

8.2 Apresentação e arguição — 6,0 pontos

Critério	Pontos	Natureza
Demonstração funcional ao vivo	1,5	do grupo
Resposta à questão sorteada (frente de outro colega)	3,5	individual
Precisão técnica, terminologia e tempo	1,0	individual

Os 4,5 pontos individuais fazem com que dois integrantes do mesmo grupo terminem com notas de A2 diferentes se um dominar o projeto inteiro e o outro apenas a própria parte. É assim que o carona é identificado: não pela entrega, que é do grupo, mas pela arguição, que é individual.

Penalidades: estouro de tempo (até −0,5); demonstração que não roda ao vivo (−1,5 no critério de demonstração); commit único ou trabalho concentrado numa só pessoa (até −1,0 no artefato); não saber explicar a frente questionada zera o critério individual correspondente.

9. Integridade Acadêmica e Uso de IA

O código pode ser desenvolvido com o apoio que você quiser, inclusive ferramentas de inteligência artificial. Não há proibição de usar IA para escrever ou revisar código paralelo — isso faz parte do trabalho de um profissional hoje.

O que é individual e sem apoio é a defesa. Na arguição, você responde sozinho, consultando apenas o próprio repositório, sem colegas e sem IA. Um artefato impecável que o autor não sabe explicar zera o critério de arguição correspondente — e é por isso que a apresentação vale mais que a entrega. No mercado, ninguém vai te impedir de usar as ferramentas que existem, mas você precisa entender o que entrega, porque será você quem vai defender aquilo, depurar e responder pelo que quebrou.

10. Observações Importantes

- Projetos superficiais, sem justificativas e sem análise crítica, não serão considerados suficientes. Um speedup relatado sem gráfico e sem discussão das causas não conta;
- A ausência na apresentação implica nota zero no seminário, impactando diretamente a nota do projeto. A frequência é registrada nas duas datas;
- Não é permitido alterar a lógica de cálculo nem a entrada do código sequencial fornecido: ele define o resultado correto. Suas versões paralelas devem chegar ao mesmo resultado;
- O ambiente de referência é o do laboratório (Linux, VS Code, g++ e OpenMPI); para casa, use o contêiner Docker ou a máquina virtual, conforme os roteiros de instalação da disciplina. Teste o ambiente antes da apresentação;
- O projeto deve demonstrar maturidade técnica, organização e coerência com os conteúdos da disciplina.

Fim do documento.